



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Lucas Pereira da Silva

Refatoração Automática de Código de Teste

Florianópolis
16 de agosto de 2026

SUMÁRIO

1	INTRODUÇÃO	2
1.1	PROBLEMA DE PESQUISA	5
1.2	HIPÓTESE DE PESQUISA	6
1.3	PROPOSTA	6
1.4	OBJETIVOS	6
1.5	ORGANIZAÇÃO DO TRABALHO	6
2	FUNDAMENTAÇÃO TEÓRICA	7
2.1	TESTE DE SOFTWARE	7
2.1.1	Categorias e estratégias de configuração de acessórios	9
2.1.1.1	<i>Estratégia configuração local</i>	9
2.1.1.2	<i>Estratégia configuração implícita</i>	10
2.1.1.3	<i>Estratégia configuração delegada</i>	11
2.1.1.4	<i>Categoria de estratégias de configuração de acessórios compartilhados</i>	11
2.2	MÉTRICAS DE SIMILARIDADE	12
2.2.1	Métricas de similaridade de código	12
2.2.1.1	<i>Métricas baseadas em medidas do código</i>	12
2.2.1.2	<i>Métricas baseadas em texto</i>	13
2.2.1.3	<i>Métricas baseadas em tokens</i>	13
2.2.1.4	<i>Métricas baseadas em árvores</i>	13
2.2.1.5	<i>Métricas baseadas em grafo</i>	14
2.3	ALGORITMOS DE AGRUPAMENTO	14
2.3.1	Métodos hierárquicos	15
2.3.2	Métodos particionais	15
2.3.3	Métodos baseados em densidade	16
2.3.4	Métodos baseados em modelo	16
2.3.5	Métodos baseados em grade	16
2.3.6	Métodos de computação suave	16
	REFERÊNCIAS	18

1 INTRODUÇÃO

Teste de software é uma das atividades presentes no processo de desenvolvimento e manutenção de software. De acordo com o *Guide to the Software Engineering Body of Knowledge* (SWEBOK) (Bourque; Fairley et al., 2014), teste de software consiste na verificação dinâmica de que um programa provê um comportamento esperado a partir de um conjunto finito de casos de teste, adequadamente selecionados a partir de um domínio de execução potencialmente infinito. Por muito tempo, teste de software foi visto sobretudo como uma etapa de verificação da qualidade do software produzido, e não como meio de construir um produto com qualidade (Murugesan, 1994). Nesse sentido, testes eram frequentemente realizados apenas ao final da etapa de codificação (Yang et al., 2011), com o objetivo de detectar falhas e fornecer um indicativo de que o software funciona conforme o esperado (Bertolino, 2007). Com o passar do tempo, a percepção sobre a utilidade do teste evoluiu. A atividade deixou de ser vista como etapa posterior ao desenvolvimento e passou a estar presente durante todo o processo de desenvolvimento e manutenção de software (Bourque; Fairley et al., 2014). Meszaros (2007) considera a atividade de teste como parte intrínseca do processo de desenvolvimento de software. Assim, a atividade de teste de software passou a incorporar propósitos variados, como detectar falhas (Tiwari; Goel, 2013), prevenir que erros sejam introduzidos (Beizer, 1990), prover um indicativo de que o software funciona (Bertolino, 2007), auxiliar na compreensão do projeto e do código do sistema (Freeman; Pryce, 2009) e servir como meio de especificação (Alvestad, 2007) e de documentação (Haugset; Hanssen, 2008) dos requisitos.

Nesse contexto, o código de teste deixou de ser um artefato auxiliar e passou a exigir os mesmos cuidados aplicados ao código de produção. Assim como o código da aplicação, o código de teste precisa ser mantido, compreendido e ajustado durante todo o projeto (Greiler; Deursen; Storey, 2013b). Para Meszaros (2007), a atividade de teste não deveria aumentar o esforço total de desenvolvimento e manutenção do software, uma vez que o esforço gasto com essa atividade deve ser compensado pela melhoria da qualidade do software. Entretanto, manter o código de teste apresenta desafios, principalmente quando a codificação e a evolução dos testes são realizadas diretamente pelo desenvolvedor. Conforme o software evolui e a quantidade de código aumenta, torna-se mais difícil manter os testes com consistência e sem perder o controle sobre eles. Um fenômeno semelhante ocorre no código de produção: sem refatoração contínua, cada nova funcionalidade tende a exigir mais esforço do que a anterior (Fowler, 1999). Com os testes, observa-se comportamento análogo. Novos testes passam a ser mais difíceis de adicionar e a duplicação tende a aumentar (Berner; Weber; Keller, 2005). Isso pode levar à degradação da manutenibilidade do código de teste (Greiler; Deursen; Storey, 2013a). Segundo Emery (2009), um motivo comum para o abandono de testes automatizados é o fato de os testes se tornarem frágeis e de alto custo de manutenção, o que pode tornar os desenvolvedores reticentes tanto em adicionar novos testes quanto em executar os existentes. Mesmo em um cenário em que tanto o código de produção quanto o código de teste são gerados por modelos de Inteligência Artificial (IA), a manutenibilidade do código de teste é um problema que pre-

cisa ser tratado. Shamshiri et al. (2018) mostra que testes gerados automaticamente afetam o tempo e a confiança do desenvolvedor, e que as tarefas de manutenção levaram 29% mais tempo quando realizadas com testes gerados automaticamente. Nesse sentido, independentemente do contexto, a manutenibilidade do código de teste ainda é um aspecto importante no processo de desenvolvimento de software.

Evitar a duplicação no código de teste é uma das formas de melhorar a manutenibilidade. Isso ocorre porque mudanças na aplicação passam a ter impacto menor nos testes, e o desenvolvedor dispõe de menos código para manter e gerenciar (Berner; Weber; Keller, 2005; Greiler; Deursen; Storey, 2013a). De acordo com Berner, Weber e Keller (2005), testes tendem a ser repetitivos e apresentam alto potencial de reuso; conforme o software evolui e o número de testes aumenta, é comum que a quantidade de código duplicado entre os testes também aumente. As Figuras 1 e 2 ilustram duplicação no código de teste. Cada figura apresenta um método de teste em uma classe distinta. Os exemplos utilizam a linguagem Kotlin e o framework JUnit 5. O código das linhas 3–8 em ambos os testes configura o sistema em teste (*System Under Test* – SUT) da mesma forma, o que produz código de configuração duplicado.

```

1  @Test
2  fun 'deve cancelar a matrícula do estudante'() {
3      val ufsc = Universidade("UFSC")
4      val alice = ufsc.admitir("Alice")
5      val bob = ufsc.contratar("Bob")
6      val cc = ufsc.criarPrograma("Ciência da Computação")
7      val algoritmos = cc.criarDisciplina("Algoritmos", bob)
8      algoritmos.maticular(alice)
9
10     val sucesso = algoritmos.cancelarMatricula(alice)
11
12     assertTrue(sucesso)
13     assertTrue(algoritmos.estudantes.isEmpty())
14 }

```

Figura 1 – Método de teste com configuração do SUT no próprio método, suscetível à duplicação de código

Fonte: Elaborado pelo autor.

Técnicas que reduzam essa duplicação podem contribuir para preservar a manutenibilidade do código de teste ao longo do projeto, bem como para diminuir o esforço total de desenvolvimento dos testes. Uma dessas técnicas é a configuração implícita (ou *implicit setup*), em que os testes são organizados em classes de teste, de forma que o código de configuração comum possa ser centralizado em um método de configuração (Meszaros, 2007). A Figura 3 ilustra a aplicação da configuração implícita. Além da remoção da duplicação de código, esse exemplo mostra como a utilização da configuração implícita traz mais clareza à compreensão do teste. O contexto comum (ter um aluno matriculado em uma disciplina) aparece apenas uma vez e cada método de teste foca no aspecto mais importante para aquele teste.

Entretanto, nem sempre é simples decidir como os testes devem ser organizados. Por exemplo, considere o teste da Figura 4. Esse teste requer um estado do SUT diferente do estado comum dos testes da Figura 3. Em vez de uma disciplina com um aluno matriculado, esse teste

```

1 @Test
2 fun 'não deve matricular o estudante duas vezes'() {
3     val ufsc = Universidade("UFSC")
4     val alice = ufsc.admitir("Alice")
5     val bob = ufsc.contratar("Bob")
6     val cc = ufsc.criarPrograma("Ciência da Computação")
7     val algoritmos = cc.criarDisciplina("Algoritmos", bob)
8     algoritmos.matricular(alice)
9
10    val sucesso = algoritmos.matricular(alice)
11
12    assertFalse(sucesso)
13    assertEquals(listOf(alice), algoritmos.estudantes)
14 }

```

Figura 2 – Método de teste com lógica de configuração do SUT idêntica à da Figura 1, ilustrando a duplicação

Fonte: Elaborado pelo autor.

```

1 class TesteUniversidade {
2     private lateinit var algoritmos: Disciplina
3     private lateinit var alice: Estudante
4
5     @BeforeEach
6     fun configurar() {
7         val ufsc = Universidade("UFSC")
8         alice = ufsc.admitir("Alice")
9         val bob = ufsc.contratar("Bob")
10        val cc = ufsc.criarPrograma("Ciência da Computação")
11        algoritmos = cc.criarDisciplina("Algoritmos", bob)
12        algoritmos.matricular(alice)
13    }
14
15    @Test
16    fun 'deve cancelar a matrícula do estudante'() {
17        val sucesso = algoritmos.cancelarMatricula(alice)
18
19        assertTrue(sucesso)
20        assertTrue(algoritmos.estudantes.isEmpty())
21    }
22
23    @Test
24    fun 'não deve matricular o estudante duas vezes'() {
25        val sucesso = algoritmos.matricular(alice)
26
27        assertFalse(sucesso)
28        assertEquals(listOf(alice), algoritmos.estudantes)
29    }
30 }

```

Figura 3 – Suíte de testes refatorada com configuração implícita, centralizando a lógica comum de inicialização e eliminando a duplicação

Fonte: Elaborado pelo autor.

requer uma disciplina fechada e sem alunos matriculados. Como nem toda configuração pode ser compartilhada, o novo teste não poderia ser colocado na mesma classe de teste dos demais sem que o código da classe existente fosse adaptado. A pergunta que fica é: compensaria fazer essa refatoração? Por um lado, o código que é comum ao novo teste e aos demais (linhas 3–7 da Figura 4) deixaria de ser duplicado. Por outro lado, o código que realiza a matrícula

do aluno na disciplina (linha 12 da Figura 3) precisaria ser duplicado nos dois testes que a exigem. Considerando apenas o reuso de código, é tácito concluir que compensaria colocar o novo teste na mesma classe de teste dos demais, uma vez que o reuso obtido seria maior que a duplicação introduzida; para reusar cinco linhas de código, uma linha de código precisaria ser duplicada duas vezes. Mas, e se ao invés de apenas dois, a classe existente já tivesse seis testes? Nesse caso, mover o novo teste para a mesma classe dos demais testes introduziria mais linhas duplicadas do que aquelas que seriam reusadas.

```

1  @Test
2  fun 'não deve matricular estudante em disciplina fechada'() {
3      val ufsc = Universidade("UFSC")
4      val alice = ufsc.admitir("Alice")
5      val bob = ufsc.contratar("Bob")
6      val cc = ufsc.criarPrograma("Ciência da Computação")
7      val algoritmos = cc.criarDisciplina("Algoritmos", bob)
8      algoritmos.fechar()
9
10     val sucesso = algoritmos.matricular(alice)
11
12     assertFalse(sucesso)
13     assertTrue(algoritmos.estudantes.isEmpty())
14 }

```

Figura 4 – Método de teste que requer um estado distinto do SUT, impedindo o reuso da configuração comum da Figura 3

Fonte: Elaborado pelo autor.

O problema fica evidente mesmo com um exemplo simples: organizar os métodos de teste levando em conta a redução da duplicação de código não é uma tarefa trivial. Em especial, em projetos com centenas ou milhares de testes, a distribuição de um novo teste leva a uma explosão combinatória de possibilidades. Convém destacar que não é apenas uma decisão sobre qual é a melhor classe de teste para inserir o novo teste, mas também sobre como re-distribuir os testes já existentes entre as classes de teste da maneira mais eficiente. Quando um novo teste é introduzido, por exemplo, é possível que a nova distribuição ótima acabe separando testes que antes pertenciam à mesma classe de teste. Percebe-se, então, que com o objetivo de atingir uma distribuição ótima dos testes considerando a redução da duplicação de código, é necessário que o código de teste seja continuamente refatorado e reorganizado.

1.1 PROBLEMA DE PESQUISA

Diante desse contexto, o problema de pesquisa é enunciado da seguinte maneira: como reduzir a duplicação de código de teste sem aumentar o esforço total de desenvolvimento e manutenção e sem alterar o resultado dos testes?

1.2 HIPÓTESE DE PESQUISA

Considerando o problema apresentado, a hipótese de pesquisa deste trabalho é enunciada da seguinte forma: é possível reduzir a duplicação de código de teste e o esforço de desenvolvimento por meio de um método de refatoração automática que trate o reuso de código de teste como um problema de otimização global da distribuição de métodos entre classes de teste.

1.3 PROPOSTA

Em linha com a hipótese apresentada, este trabalho propõe a utilização de métricas de similaridade e algoritmos de agrupamento para resolver o problema de otimização global da distribuição de métodos entre classes de teste. As métricas de similaridade permitem identificar testes com código de configuração em comum; os algoritmos de agrupamento, por sua vez, propõem grupos de testes cuja redistribuição maximiza o reuso de código pela estratégia de configuração implícita.

1.4 OBJETIVOS

O objetivo geral deste trabalho consiste em reduzir a duplicação de código de teste e o esforço total de desenvolvimento e manutenção dos testes, sem alterar o resultado dos testes. Para alcançar o objetivo geral, os seguintes objetivos específicos deverão ser alcançados: (1) investigar métricas de similaridade para identificar código de configuração em comum entre testes; (2) investigar algoritmos de agrupamento para propor distribuições de métodos entre classes de teste; (3) propor uma arquitetura em pipeline para refatoração de código de teste; (4) especificar e implementar um framework que instancia essa arquitetura sem alterar o comportamento observado dos testes; e (5) avaliar a abordagem quanto à redução de duplicação e ao esforço de refatoração.

1.5 ORGANIZAÇÃO DO TRABALHO

Os demais capítulos deste trabalho são organizados da seguinte forma: no Capítulo 2 é apresentada a fundamentação teórica; no Capítulo 3 é apresentado o estado da arte e os trabalhos relacionados; no Capítulo 4 é apresentada a proposta deste trabalho; e no Capítulo 5 é apresentada a conclusão e os próximos passos para elaboração deste trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

Três tópicos se destacam em nível de importância para este trabalho: teste de software, métricas de similaridade e algoritmos de agrupamento. Neste capítulo são apresentados os principais conceitos envolvendo a implementação de testes automatizados, as diferentes famílias de métricas de similaridade de código e os métodos de agrupamento utilizados para formar grupos de elementos similares.

2.1 TESTE DE SOFTWARE

Um teste é um procedimento, manual ou automático, que pode ser usado para verificar se um dado sistema em teste (*System Under Test* – SUT) funciona de acordo com o comportamento esperado. O SUT representa a parte do sistema que está sendo testada. Cada teste deve realizar, inicialmente, uma série de passos para colocar o SUT no estado desejado para que o teste possa ser executado. Os elementos necessários para que o SUT seja colocado no estado desejado recebem o nome de *test fixtures*, ou simplesmente *fixtures*, ou acessórios de teste. A parte da lógica do teste em que os acessórios são criados é chamada de *fixture setup*, ou configuração de acessórios. Após executar a configuração de acessórios, o teste exercita o SUT através da execução da operação que está sendo testada, realiza uma série de verificações para garantir que o SUT funciona de acordo com o comportamento esperado e, por fim, limpa os acessórios.

Os exemplos apresentados neste trabalho são baseados no framework de teste JUnit, disponível para as linguagens Java e Kotlin. Os exemplos utilizam a linguagem Kotlin. A Figura 5 apresenta uma classe de teste com dois métodos de teste. A anotação `@Test` nas linhas 2 e 12 indica ao framework que os métodos anotados correspondem a testes. Essa abordagem segue a definição utilizada por Freeman e Pryce (2009), na qual, tipicamente, cada teste é separado em um método de teste. Além disso, cada teste pode ser dividido conceitualmente em quatro fases (Meszaros, 2007): configuração (*setup*), exercício, verificação e finalização (*tear down*).

Considerando o primeiro teste da Figura 5, as linhas 4 e 5 correspondem à etapa de configuração. Nessa etapa, o SUT deve ser colocado no estado desejado para o teste. É na configuração que os acessórios necessários para o teste são definidos. Nesse caso, a instância da classe `RepositorioDeUsuario` é um acessório necessário para o teste, e a conexão com o banco de dados é estabelecida na linha 5. A etapa seguinte consiste em exercitar o SUT. É nessa etapa que é executado o que efetivamente será testado. Na Figura 5, a linha 6 corresponde à etapa de exercício do primeiro teste. Na etapa de verificação, o comportamento esperado do sistema é verificado a partir das saídas produzidas pelo SUT ou através de mecanismos que permitam obter o estado interno do SUT (e.g., consultas no banco de dados). No primeiro teste da Figura 5, a verificação ocorre na linha 7. A última etapa consiste em limpar os acessórios definidos na configuração, para evitar que futuras execuções do teste, ou que execuções futuras de outros testes, venham a ser afetadas. No primeiro teste da Figura 5, a finalização ocorre nas

```

1 class TesteRepositorioDeUsuario {
2     @Test
3     fun deveInserirUsuario() {
4         val repositorio = RepositorioDeUsuario()
5         repositorio.conectar()
6         val inserido = repositorio.inserir("Alice")
7         assertTrue(inserido)
8         repositorio.remover("Alice")
9         repositorio.desconectar()
10    }
11
12    @Test
13    fun deveRemoverUsuario() {
14        val repositorio = RepositorioDeUsuario()
15        repositorio.conectar()
16        repositorio.inserir("Alice")
17        val removido = repositorio.remover("Alice")
18        assertTrue(removido)
19        repositorio.desconectar()
20    }
21 }

```

Figura 5 – Classe de teste com dois métodos que exercitam o repositório de usuários, ilustrando as etapas de configuração, exercício, verificação e finalização

Fonte: Elaborado pelo autor.

linhas 8 e 9, com a remoção do usuário inserido e o encerramento da conexão com o banco de dados.

De acordo com Meszaros (2007), existem estratégias de agrupamento para organizar os testes em classes. Na estratégia *testcase class per feature*, ou classe de teste por funcionalidade da aplicação, os testes que verificam uma mesma funcionalidade, ou um mesmo requisito, são reunidos em uma classe, independentemente da classe de produção exercitada. O objetivo é facilitar a leitura dos testes como especificação daquela funcionalidade. Na estratégia *testcase class per class*, ou classe de teste por classe da aplicação, cada classe de produção possui uma classe de teste correspondente, que concentra os testes dos seus métodos. Essa organização simplifica a correspondência entre o código de produção e o código de teste. Entretanto, os testes da mesma classe podem exigir acessórios distintos entre si. Na estratégia *testcase class per fixture*, ou classe de teste por acessório, os testes são agrupados pelo estado inicial de que necessitam. Assim, testes que utilizam acessórios semelhantes ficam na mesma classe, o que permite centralizar a configuração comum, em especial pela estratégia de configuração implícita.

O manifesto da automação de testes (Meszaros; Smith; Andrea, 2003) apresenta características esperadas dos testes automatizados. Os testes devem ser autoverificáveis (i.e., cada teste reporta o próprio resultado); repetíveis, de modo que possam ser executados múltiplas vezes; robustos, produzindo resultados consistentes para o mesmo código de aplicação; e independentes, permitindo que cada teste seja executado isoladamente ou em conjunto com outros testes, em qualquer ordem. Inovações na atividade de teste devem preservar essas qualidades.

2.1.1 Categorias e estratégias de configuração de acessórios

A configuração de acessórios compreende a lógica do teste em que os acessórios são criados. Cada teste possui uma configuração de acessórios, que consiste na criação dos acessórios que serão utilizados no teste. A configuração de acessórios de um teste pode ser realizada através de diferentes estratégias, podendo ser uma ou mais estratégias ao mesmo tempo. Um dos problemas que afetam a manutenibilidade do código de teste é a duplicação do código de configuração de acessórios. Nem sempre é simples ou possível evitar esse tipo de problema, porém existem diferentes estratégias de configuração de acessórios que podem ser utilizadas para reduzir a duplicação de código.

Meszaros (2007) divide as estratégias de configuração de acessórios em duas categorias: *fresh fixture setup*, ou configuração de acessórios novos, e *shared fixture construction*, ou configuração de acessórios compartilhados. Em ambas as categorias existem estratégias que possibilitam o reuso de código. Entretanto, apenas na configuração de acessórios compartilhados é possível o reuso de execução. A categoria configuração de acessórios novos é composta por três estratégias: *inline setup*, ou configuração local; *implicit setup*, ou configuração implícita; e *delegated setup*, ou configuração delegada. Nessa categoria há apenas a possibilidade de reuso de código, mas não de reuso de execução. Já as estratégias da configuração de acessórios compartilhados realizam a criação de acessórios que podem ser criados uma única vez e reutilizados entre várias execuções de teste distintas.

2.1.1.1 Estratégia configuração local

Na estratégia configuração local, os acessórios são criados dentro do próprio método de teste. Em geral, essa estratégia é utilizada em testes que necessitam de acessórios muito específicos ou no desenvolvimento dos primeiros testes, em que ainda não existe a necessidade de reuso de acessórios. A Figura 6 mostra um exemplo de teste em que a estratégia configuração local é utilizada. As linhas 4 e 5 contêm a configuração de acessórios do teste. Destaca-se que, na estratégia configuração local, a configuração de acessórios é realizada diretamente no método de teste.

```

1 class TesteCriarBancoComConfiguracaoLocal {
2     @Test
3     fun criarBanco() {
4         val sistemaBancario = SistemaBancario()
5         val bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
6         assertEquals("Banco do Brasil", bb.obterNome())
7         assertEquals(Moeda.BRL, bb.obterMoeda())
8     }
9 }

```

Figura 6 – Método de teste com configuração local, na qual os acessórios são criados no próprio método

Como os acessórios são criados diretamente no método de teste, torna-se fácil compreender a relação de causa e efeito entre os acessórios e as saídas do SUT. Entretanto, a longo prazo, essa estratégia tende a aumentar a duplicação de código, uma vez que diversos testes podem utilizar acessórios idênticos ou muito similares. Além disso, essa estratégia pode dificultar a compreensão do teste nos casos em que os acessórios forem muito complexos, uma vez que a quantidade de código aumenta e se torna mais difícil compreender o papel do acessório no teste.

2.1.1.2 Estratégia configuração implícita

Na estratégia configuração implícita, os acessórios são criados através de um método especial, comumente denominado de *setup method*, ou método de configuração, pertencente à classe de teste e executado antes de cada método de teste. A Figura 7 apresenta um exemplo da estratégia configuração implícita. Nesse exemplo, o método configurar da linha 5 representa o método de configuração. O atributo bb declarado na linha 2 é utilizado para permitir que o acessório seja manuseável pelos testes da classe. Quando um teste da classe é executado, o framework fica responsável por descobrir e executar o método de configuração. No JUnit, a anotação @BeforeEach é utilizada para indicar o método de configuração.

```

1 class TesteCriarBancoComConfiguracaoImplicita {
2     private lateinit var bb: Banco
3
4     @BeforeEach
5     fun configurar() {
6         val sistemaBancario = SistemaBancario()
7         bb = sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
8     }
9
10    @Test
11    fun criarBanco() {
12        assertEquals("Banco do Brasil", bb.obterNome())
13        assertEquals(Moeda.BRL, bb.obterMoeda())
14    }
15 }

```

Figura 7 – Classe de teste com configuração implícita, na qual os acessórios são criados em um método de configuração executado antes de cada teste

Fonte: Elaborado pelo autor.

A vantagem dessa estratégia é promover o reuso de acessórios entre testes de uma classe. Porém, para utilizar essa estratégia, os testes que utilizam os mesmos acessórios devem ser agrupados na mesma classe. Isso pode trazer uma complicação a longo prazo, pois se torna cada vez mais difícil agrupar testes que necessitem exatamente dos mesmos acessórios. Com facilidade, alguns testes irão necessitar de acessórios específicos somente para eles. Além disso, nem sempre é possível evitar a duplicação de código através dessa estratégia, uma vez que um determinado teste pode ter acessórios similares a dois outros testes que, por sua vez, não possuam acessórios similares entre si.

Greiler, Deursen e Storey (2013a) apresenta *test smells*, ou fedores de teste, que podem dificultar a manutenibilidade do código de teste quando essa estratégia é utilizada. Um dos principais problemas apontados é o *general fixture smell*, ou fedor de acessórios generalizados, que ocorre quando existem acessórios dentro do método de configuração que são necessários apenas para alguns testes da classe. Isso pode prejudicar a legibilidade dos demais testes, uma vez que existirão mais acessórios do que apenas os necessários.

2.1.1.3 Estratégia configuração delegada

A estratégia configuração delegada utiliza métodos auxiliares responsáveis pela criação dos acessórios. Em geral, esses métodos são disponibilizados através de classes auxiliares, separadas das classes de teste. A Figura 8 apresenta um exemplo da estratégia configuração delegada. O acessório necessário para o teste é criado através do método `criarBancoDoBrasil`, definido na linha 11. Esse método auxiliar é chamado na linha 4 da classe de teste.

```

1 class TesteCriarBancoComConfiguracaoDelegada {
2     @Test
3     fun criarBanco() {
4         val bb = Auxiliar.criarBancoDoBrasil()
5         assertEquals("Banco do Brasil", bb.obterNome())
6         assertEquals(Moeda.BRL, bb.obterMoeda())
7     }
8 }
9
10 object Auxiliar {
11     fun criarBancoDoBrasil(): Banco {
12         val sistemaBancario = SistemaBancario()
13         return sistemaBancario.criarBanco("Banco do Brasil", Moeda.BRL)
14     }
15 }

```

Figura 8 – Método de teste com configuração delegada, na qual a criação dos acessórios é delegada a um método auxiliar

Fonte: Elaborado pelo autor.

A principal vantagem dessa estratégia é permitir que o mesmo código de teste possa ser utilizado por testes diferentes de classes diferentes. A desvantagem dessa estratégia é que, em alguns casos, a sua utilização pode dificultar a compreensão da relação de causa e efeito entre os acessórios e as saídas do SUT, uma vez que o método auxiliar é, comumente, colocado em uma classe separada do teste.

2.1.1.4 Categoria de estratégias de configuração de acessórios compartilhados

As estratégias da categoria configuração de acessórios compartilhados realizam a criação de acessórios coletivos. Dessa forma, um acessório pode ser criado uma única vez e reutilizado entre várias execuções de teste distintas. Um exemplo descrito por Meszaros (2007) é a estratégia *lazy setup*, ou configuração preguiçosa, na qual o primeiro teste cria o acessório

e os testes seguintes reutilizam a mesma instância, o que é útil quando a criação do acessório é custosa (e.g., a população de um banco de dados compartilhado entre os testes de uma suíte). Os frameworks de teste atuais não oferecem formas nativas e nem padronizadas para realizar esse tipo de configuração de acessórios. Por isso, é necessário que o desenvolvedor do teste assuma esse papel e passe a gerenciar parte do fluxo de execução dos testes.

2.2 MÉTRICAS DE SIMILARIDADE

Métricas de similaridade possuem aplicações em diversas áreas da computação, como reconhecimento facial em imagens (Cao; Ying; Li, 2013), busca textual (Sahami; Heilman, 2006) e recuperação de informação estruturada (Dorneles et al., 2004). Métricas de similaridade determinam o grau de similaridade entre dois elementos, em que o grau de similaridade é um valor entre 0 e 1 que indica o quanto dois elementos são parecidos entre si de acordo com o critério definido pela métrica. Cada métrica de similaridade possui um mecanismo para gerar o grau de similaridade, de modo que cada métrica apresenta uma distribuição específica sobre o domínio de valores. Neste trabalho, pretende-se avaliar a similaridade entre testes através do código de teste. Por isso, métricas de similaridade de código são abordadas a seguir.

2.2.1 Métricas de similaridade de código

Avaliar a similaridade de código é uma atividade que possui propósitos variados, como identificação de fragmentos duplicados de código (Roy; Cordy, 2008), detecção de plágio e verificação de violação de direitos de cópia de software (Ottenstein, 1976), recomendação de código (Holmes; Murphy, 2005) e outros. Ragkhitwetsagul, Krinke e Clark (2018) classificam cinco diferentes abordagens de métricas de similaridade de código: baseadas em medidas do código, baseadas em texto, baseadas em tokens, baseadas em árvore e baseadas em grafo. Nas seções a seguir, essas cinco abordagens são apresentadas.

2.2.1.1 Métricas baseadas em medidas do código

Medidas do código como a quantidade de linhas e a quantidade de variáveis entre duas funções diferentes, podem ser usadas como indicativos de similaridade de código. As abordagens baseadas em medidas do código foram as primeiras relatadas na literatura e foram aplicadas na detecção de similaridade de software. Ottenstein (1976) utiliza métricas de similaridade para identificar potenciais equivalências entre programas de alunos de uma disciplina de programação. A técnica contabiliza o número de operandos e operadores, com base na heurística de que a probabilidade de dois programas distintos apresentarem as mesmas contagens de operandos e operadores é baixa. Com isso, a métrica é robusta à renomeação de variáveis.

O problema de identificação de plágios entre trabalhos de disciplinas de programação também é abordado por Donaldson, Lancaster e Sposato (1981). Nessa abordagem, o número

de declarações, chamadas, leituras, atribuições, comandos `while` e `if`, e definições de funções é contabilizado para avaliar a similaridade entre programas, permitindo a detecção de semelhanças estruturais.

2.2.1.2 *Métricas baseadas em texto*

Métricas de similaridade de código baseadas em texto avaliam a similaridade entre dois programas através da análise de sequências de caracteres do código-fonte. Roy e Cordy (2008) propõem a identificação de clones de fragmentos de código através de uma técnica que analisa diretamente o código-fonte. Diferente de abordagens que buscam identificar plágio, a proposta busca encontrar fragmentos de código intencionalmente copiados com o propósito de reuso. A abordagem representa o código como sequências de linhas e computa a similaridade utilizando o algoritmo da subsequência comum mais longa (*Longest Common Subsequence* – LCS) (Hirschberg, 1977). Antes de converter o código em sequências de linhas, a abordagem normaliza o código-fonte, removendo indentação e comentários e padronizando açúcar sintático.

Em outro trabalho, a similaridade entre dois programas é determinada através de uma matriz em que cada linha de um programa é comparada com todas as linhas do outro (Ducasse; Rieger; Demeyer, 1999). Quando duas linhas são idênticas, a entrada correspondente da matriz recebe o valor um; caso contrário, recebe o valor zero.

2.2.1.3 *Métricas baseadas em tokens*

Assim como as abordagens baseadas em texto, as abordagens baseadas em tokens também avaliam a similaridade entre dois programas através da análise textual do código-fonte. Abordagens baseadas em tokens, no entanto, aumentam o nível de abstração convertendo o código-fonte em sequências de tokens. A utilização de tokens visa normalizar diferenças textuais através da definição de tipos abstratos de constructos textuais (e.g., todos os identificadores ou valores literais presentes no código podem ser representados por um mesmo token).

Kamiya, Kusumoto e Inoue (2002) utilizam a abordagem baseada em token na ferramenta CCFinder. A ferramenta é capaz de identificar clones em diferentes linguagens, incluindo C, C++, Java e COBOL. Para cada linguagem suportada, um analisador léxico dedicado é utilizado para tokenizar o código-fonte. O método busca descobrir classes de clones (i.e., fragmentos de código que compartilham uma sequência idêntica de tokens). O resultado é a identificação e extração de rotinas comuns, incentivando o reuso de código.

2.2.1.4 *Métricas baseadas em árvores*

Diferente das categorias vistas anteriormente, esta categoria de técnicas tem o foco voltado para identificar a similaridade entre dois programas com base na semelhança estrutural entre eles. Nesse sentido, diferenças léxicas são menos relevantes na análise de similaridade.

Para avaliar a similaridade entre dois programas são construídas árvores abstratas de sintaxe (*Abstract Syntax Tree* – AST). As ASTs permitem identificar a similaridade entre dois trechos mesmo com uma alta quantidade de modificações entre eles. A complexidade computacional, entretanto, é mais alta quando comparada com as categorias de técnicas vistas anteriormente.

Baxter et al. (1998) utilizam ASTs para identificar clones, ou clones que sofreram modificações, no contexto de programas escritos na linguagem C. O objetivo do trabalho é identificar clones criados intencionalmente, em que um desenvolvedor copia um fragmento existente para construir um novo fragmento semelhante. A detecção automática alerta os desenvolvedores para esses clones, de modo que a duplicação possa ser eliminada via refatoração. A partir das ASTs, a similaridade é computada dividindo o número de nós coincidentes pelo número total de nós. Um limiar de similaridade é então aplicado, e fragmentos cuja similaridade ultrapassa esse limiar são reportados como clones.

2.2.1.5 Métricas baseadas em grafo

Árvores podem ser vistas como uma especialização de grafos. Nesse sentido, métricas de similaridade baseadas em árvores se assemelham às métricas baseadas em grafos, uma vez que ambas permitem capturar aspectos estruturais do código-fonte. As métricas baseadas em grafo, no entanto, possuem como característica adicional a capacidade de identificar similaridade de aspectos semânticos. Essa capacidade adicional ocorre ao custo de maior complexidade computacional, inerente aos algoritmos sobre grafos.

Krinke (2001) utiliza grafos para representar a estrutura e o fluxo de dados de programas e, a partir disso, identificar duplicação de código. O objetivo é maximizar a identificação de trechos de código duplicados ainda que exista diferença entre os trechos, minimizando a ocorrência de falsos positivos (i.e., evitar a identificação de dois trechos de código como duplicados quando eles não são semanticamente equivalentes). Para detectar similaridade semântica, o autor utiliza grafos de dependência de programa (*Program Dependence Graph* – PDG) (Ferrante; Ottenstein; Warren, 1987), em que os nós correspondem a comandos no código e as arestas representam dependências de dados e de fluxo de controle. Komondoor e Horwitz (2001) também representam o código-fonte através de PDGs para identificar pares de clones. Ao aplicar algoritmos de detecção de subgrafos isomorfos, a abordagem pode encontrar clones não contíguos, clones com comandos permutados e até clones interdependentes.

2.3 ALGORITMOS DE AGRUPAMENTO

Agrupamento consiste na tarefa de atribuir padrões a grupos, ou *clusters*, de maneira não supervisionada (Jain; Murty; Flynn, 1999). Técnicas de agrupamento são empregadas para formar grupos de elementos mutuamente similares. O objetivo principal é maximizar a similaridade entre elementos do mesmo grupo (similaridade intragrupo) e minimizar a similaridade entre elementos pertencentes a grupos diferentes (similaridade intergrupo). Consequentemente,

para uma coleção de entrada de elementos, uma técnica de agrupamento deve gerar, como saída, um conjunto de grupos, cada um contendo um ou mais elementos.

Há uma ampla variedade de métodos de agrupamento. Esses métodos podem ser organizados em seis categorias principais (Rokach; Maimon, 2005): hierárquicos, particionais, baseados em densidade, baseados em modelo, baseados em grade e métodos de computação suave. Cada categoria é caracterizada por um princípio de indução subjacente que conduz a operação dos algoritmos de agrupamento nela contidos.

2.3.1 Métodos hierárquicos

O agrupamento hierárquico forma grupos de maneira recursiva, seja fundindo sucessivamente agrupamentos em grupos maiores, seja dividindo-os em grupos menores. Um método hierárquico aglomerativo comumente citado inicia com n grupos, cada um contendo um único elemento (Johnson; Wichern, 2002). Em seguida, a cada passo, os dois grupos mais similares são combinados. Esse procedimento iterativo continua até que um critério de parada predefinido seja atingido.

A estrutura resultante dos grupos pode ser representada por um diagrama bidimensional em forma de árvore, denominado dendrograma (Sibbald; Wentzell; Wade, 1989). O dendrograma descreve a sequência de fusões realizadas em diferentes níveis da hierarquia. Em cada nível, os grupos podem ser fundidos utilizando uma de três estratégias de ligação: ligação simples, em que a similaridade se baseia no par de elementos mais próximos entre os grupos; ligação completa, que se apoia na maior distância entre elementos de grupos diferentes; e ligação média, que utiliza a distância média entre todos os pares de elementos dos dois grupos para quantificar a similaridade.

2.3.2 Métodos particionais

Métodos particionais, como K-means (Hartigan; Wong, 1979) e CLARANS (Ng; Han, 2002), partem de um conjunto predefinido de grupos e utilizam realocação iterativa para mover elementos de um grupo para outro, com o objetivo de atingir otimalidade global. No K-means, o número de grupos k é informado a priori. Cada grupo é representado por um centróide, calculado como a média dos elementos a ele associados, e cada elemento é atribuído ao centróide mais próximo. Os centróides são então atualizados e o processo de atribuição se repete até que as associações se estabilizem. O CLARANS segue a mesma ideia de partição em k grupos, porém representa cada grupo por um medóide (i.e., um elemento efetivamente presente no conjunto) e explora o espaço de soluções por amostragem aleatória, o que o torna adequado a conjuntos volumosos de dados espaciais.

2.3.3 Métodos baseados em densidade

Métodos baseados em densidade consideram que os grupos correspondem a regiões densas do espaço de dados, separadas por regiões de baixa densidade. O algoritmo DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) (Ester et al., 1996) é um exemplo conhecido dessa categoria. Nesse algoritmo, um ponto é considerado central quando possui um número mínimo de vizinhos em um raio ϵ ; pontos alcançáveis a partir de um ponto central pertencem ao mesmo grupo, e pontos isolados são tratados como ruído. Diferentemente dos métodos particionais, o DBSCAN não exige que o número de grupos seja definido de antemão e é capaz de identificar grupos de formato arbitrário.

2.3.4 Métodos baseados em modelo

Métodos baseados em modelo partem de um modelo matemático para gerar hipóteses e, em seguida, tentam ajustar os dados de agrupamento a esse modelo. Além de descobrir grupos, esses métodos também podem fornecer descrições características para cada grupo. Em geral, são agrupados em dois tipos principais: métodos de árvore de decisão e métodos de redes neurais, como o mapa auto-organizável (*Self-Organizing Map* – SOM) (Kohonen, 2013). Nos métodos de árvore de decisão, o espaço de atributos é particionado de forma recursiva, de modo que cada grupo passa a ser descrito pelas condições que levam até o nó correspondente. No SOM, os elementos são projetados sobre uma grade de neurônios de baixa dimensionalidade; neurônios vizinhos ajustam seus vetores de peso em direção aos padrões apresentados, de forma que regiões contíguas do mapa passam a representar grupos de elementos similares.

2.3.5 Métodos baseados em grade

Métodos baseados em grade, como STING (*Statistical Information Grid*) (Wang; Yang; Muntz, 1997), derivam grupos a partir de um conjunto finito de células dispostas em uma estrutura de grade. O espaço de dados é dividido em células retangulares em múltiplos níveis de resolução, e cada célula armazena estatísticas resumidas dos elementos que contém (e.g., contagem, média e extremos). O agrupamento opera sobre essas estatísticas, sem percorrer novamente cada elemento. O principal benefício desses métodos é a alta velocidade de processamento.

2.3.6 Métodos de computação suave

Métodos de computação suave incluem agrupamento fuzzy, recozimento simulado (*simulated annealing* – SA) (Kirkpatrick; Jr; Vecchi, 1983) e outras técnicas de otimização estocástica. No agrupamento fuzzy, um elemento pode pertencer a mais de um grupo simultaneamente, com graus de pertinência que quantificam a força dessa associação. O recozimento

simulado trata o agrupamento como um problema de otimização: partições candidatas são perturbadas de forma aleatória e, no início do processo, soluções piores podem ser aceitas para escapar de mínimos locais; a probabilidade de aceitar uma solução pior diminui gradualmente, à medida que um parâmetro análogo à temperatura é reduzido.

REFERÊNCIAS

- ALVESTAD, K. **Domain Specific Languages for Executable Specifications**. Dissertação (Mestrado) — Institutt for datateknikk og informasjonsvitenskap, 2007.
- BAXTER, I. D. et al. Clone detection using abstract syntax trees. In: **Proceedings International Conference on Software Maintenance**. [S.l.: s.n.], 1998. p. 368–377.
- BEIZER, B. **Software Testing Techniques**. 2. ed. [S.l.]: Van Nostrand Reinhold Company, 1990.
- BERNER, S.; WEBER, R.; KELLER, R. K. Observations and lessons learned from automated testing. In: **Proceedings of the 27th International Conference on Software Engineering**. [S.l.]: ACM, 2005. p. 571–579.
- BERTOLINO, A. Software testing research: Achievements, challenges, dreams. In: **2007 Future of Software Engineering**. [S.l.]: IEEE Computer Society, 2007. p. 85–103.
- BOURQUE, P.; FAIRLEY, R. E. et al. **Guide to the software engineering body of knowledge (SWEBOK): Version 3.0**. [S.l.]: IEEE Computer Society Press, 2014.
- CAO, Q.; YING, Y.; LI, P. Similarity metric learning for face recognition. In: **Proceedings of the IEEE International Conference on Computer Vision**. [S.l.: s.n.], 2013. p. 2408–2415.
- DONALDSON, J. L.; LANCASTER, A.-M.; SPOSATO, P. H. A plagiarism detection system. **SIGCSE Bull.**, v. 13, n. 1, p. 21–25, 1981.
- DORNELES, C. F. et al. Measuring similarity between collection of values. In: **Proceedings of the 6th annual ACM international workshop on Web information and data management**. [S.l.: s.n.], 2004. p. 56–63.
- DUCASSE, S.; RIEGER, M.; DEMEYER, S. A language independent approach for detecting duplicated code. In: **Proceedings IEEE International Conference on Software Maintenance**. [S.l.: s.n.], 1999. p. 109–118.
- EMERY, D. H. Writing maintainable automated acceptance tests. In: **Agile Testing Workshop, Agile Development Practices**. Orlando, Florida: [s.n.], 2009.
- ESTER, M. et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In: **Proceedings of the Second International Conference on Knowledge Discovery and Data Mining**. [S.l.: s.n.], 1996. p. 226–231.
- FERRANTE, J.; OTTENSTEIN, K. J.; WARREN, J. D. The program dependence graph and its use in optimization. **ACM Transactions on Programming Languages and Systems**, v. 9, n. 3, p. 319–349, 1987.
- FOWLER, M. **Refactoring: Improving the Design of Existing Code**. [S.l.]: Addison-Wesley, 1999.
- FREEMAN, S.; PRYCE, N. **Growing Object-Oriented Software, Guided by Tests**. [S.l.]: Pearson Education, 2009.

- GREILER, M.; DEURSEN, A. van; STOREY, M.-A. Automated detection of test fixture strategies and smells. In: **2013 IEEE Sixth International Conference on Software Testing, Verification and Validation**. [S.l.]: IEEE, 2013. p. 322–331.
- GREILER, M.; DEURSEN, A. van; STOREY, M.-A. Strategies for avoiding text fixture smells during software evolution. In: **Proceedings of the 10th Working Conference on Mining Software Repositories**. [S.l.]: IEEE Press, 2013. p. 387–396.
- HARTIGAN, J. A.; WONG, M. A. Algorithm as 136: A k-means clustering algorithm. **Journal of the Royal Statistical Society Series C**, v. 28, n. 1, p. 100–108, 1979.
- HAUGSET, B.; HANSEN, G. K. Automated acceptance testing: A literature review and an industrial case study. In: **Agile 2008 Conference**. [S.l.]: IEEE, 2008. p. 27–38.
- HIRSCHBERG, D. S. Algorithms for the longest common subsequence problem. **Journal of the ACM**, v. 24, n. 4, p. 664–675, 1977.
- HOLMES, R.; MURPHY, G. C. Using structural context to recommend source code examples. In: **Proceedings of the 27th International Conference on Software Engineering**. [S.l.: s.n.], 2005. p. 117–125.
- JAIN, A. K.; MURTY, M. N.; FLYNN, P. J. Data clustering: a review. **ACM Computing Surveys**, v. 31, n. 3, p. 264–323, 1999.
- JOHNSON, R. A.; WICHERN, D. W. **Applied multivariate statistical analysis**. Upper Saddle River, NJ: Prentice Hall, 2002.
- KAMIYA, T.; KUSUMOTO, S.; INOUE, K. Ccfinder: a multilinguistic token-based code clone detection system for large scale source code. **IEEE Transactions on Software Engineering**, v. 28, n. 7, p. 654–670, 2002.
- KIRKPATRICK, S.; JR, C. D. G.; VECCHI, M. P. Optimization by simulated annealing. **Science**, v. 220, n. 4598, p. 671–680, 1983.
- KOHONEN, T. Essentials of the self-organizing map. **Neural Networks**, v. 37, p. 52–65, 2013.
- KOMONDOOR, R.; HORWITZ, S. Using slicing to identify duplication in source code. In: **International Static Analysis Symposium**. [S.l.]: Springer, 2001. p. 40–56.
- KRINKE, J. Identifying similar code with program dependence graphs. In: **Proceedings Eighth Working Conference on Reverse Engineering**. [S.l.: s.n.], 2001. p. 301–309.
- MESZAROS, G. **xUnit test patterns: Refactoring test code**. [S.l.]: Pearson Education, 2007.
- MESZAROS, G.; SMITH, S. M.; ANDREA, J. The test automation manifesto. In: **Extreme Programming and Agile Methods - XP/Agile Universe 2003**. [S.l.]: Springer, 2003. p. 73–81.
- MURUGESAN, S. Attitude towards testing: a key contributor to software quality. In: **Proceedings of 1994 1st International Conference on Software Testing, Reliability and Quality Assurance (STRQA'94)**. [S.l.: s.n.], 1994. p. 111–115.
- NG, R. T.; HAN, J. Clarans: a method for clustering objects for spatial data mining. **IEEE Transactions on Knowledge and Data Engineering**, v. 14, n. 5, p. 1003–1016, 2002.

- OTTENSTEIN, K. J. An algorithmic approach to the detection and prevention of plagiarism. **SIGCSE Bull.**, v. 8, n. 4, p. 30–41, 1976.
- RAGKHITWETSAGUL, C.; KRINKE, J.; CLARK, D. A comparison of code similarity analysers. **Empirical Software Engineering**, v. 23, n. 4, p. 2464–2519, 2018.
- ROKACH, L.; MAIMON, O. Clustering methods. In: **Data mining and knowledge discovery handbook**. [S.l.]: Springer, 2005. p. 321–352.
- ROY, C. K.; CORDY, J. R. Nicad: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization. In: **2008 16th IEEE International Conference on Program Comprehension**. [S.l.: s.n.], 2008. p. 172–181.
- SAHAMI, M.; HEILMAN, T. D. A web-based kernel function for measuring the similarity of short text snippets. In: **Proceedings of the 15th international conference on World Wide Web**. [S.l.: s.n.], 2006. p. 377–386.
- SHAMSHIRI, S. et al. How do automatically generated unit tests influence software maintenance? In: **2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)**. [S.l.: s.n.], 2018. p. 250–261.
- SIBBALD, D. B.; WENTZELL, P. D.; WADE, A. P. Display methods for dendrograms. **TrAC Trends in Analytical Chemistry**, v. 8, n. 8, p. 289–291, 1989. ISSN 0165-9936. Disponível em: <https://www.sciencedirect.com/science/article/pii/0165993689850629>.
- TIWARI, R.; GOEL, N. Reuse: reducing test effort. **ACM SIGSOFT Software Engineering Notes**, v. 38, n. 2, p. 1–11, 2013.
- WANG, W.; YANG, J.; MUNTZ, R. Sting: A statistical information grid approach to spatial data mining. In: **Proceedings of the 23rd International Conference on Very Large Data Bases**. [S.l.: s.n.], 1997. p. 186–195.
- YANG, Y. et al. Testqual: Conceptualizing software testing as a service. **e-Service Journal**, Indiana University Press, v. 7, n. 2, p. 46–65, 2011. ISSN 15288226, 15288234. Disponível em: <http://www.jstor.org/stable/10.2979/eservicej.7.2.46>.